

Documentation Technique

Site web Savora — vitrine et réservation en ligne (SavoraWeb)

Projet :	Site web du restaurant Savora
Épreuve :	BTS SIO – E6 (option SLAM)
Version :	1.0
Date :	Mars 2026
Organisation :	Restaurant gastronomique Savora
Candidat :	Balla Gueye — N° 02149092520

1. Présentation du site

Le site web Savora est un **site vitrine** doublé d'un **module de réservation en ligne**. Il a deux rôles :

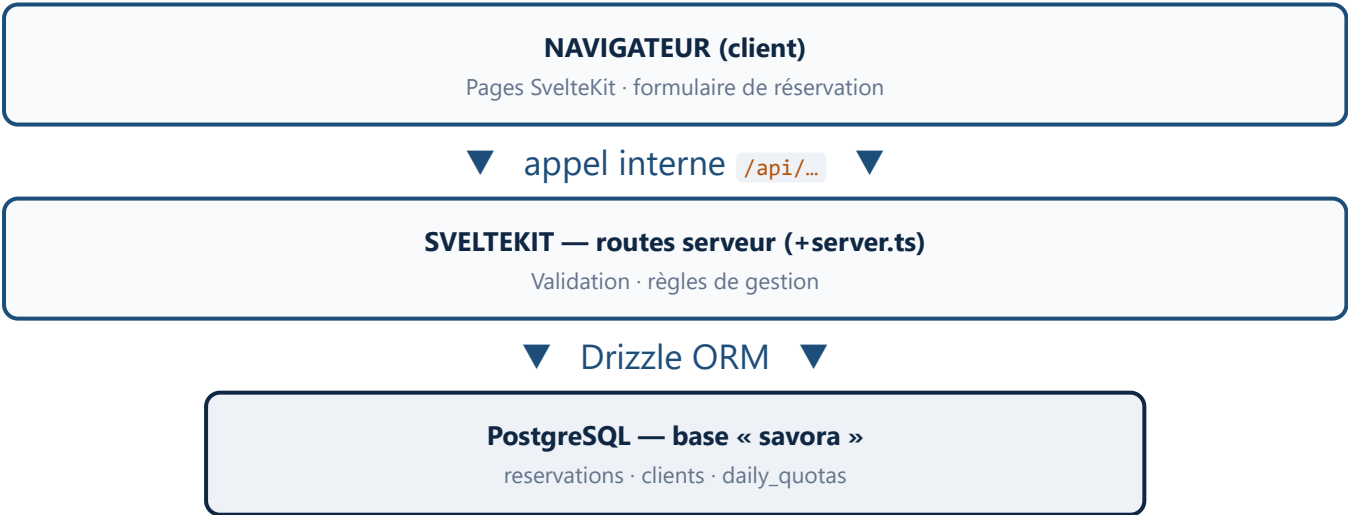
Rôle	Description
Vitrine	Présenter le restaurant : concept, philosophie, horaires, localisation
Réservation	Permettre au client de réserver une table en vérifiant les disponibilités en temps réel

1.1 Technologies utilisées

Outil	Rôle
SvelteKit	Framework du site (pages côté navigateur + routes côté serveur)
Svelte 5	Affichage réactif (runes : <code>\$state</code> , <code>\$effect</code>)
Drizzle ORM	Accès à la base PostgreSQL (en TypeScript, typé et sécurisé)
PostgreSQL	Base de données (réservations, clients, quotas)
TypeScript	Langage typé (évite les erreurs)

1.2 Schéma de l'architecture

SvelteKit est un framework **full-stack** : il sert les pages au navigateur et exécute aussi du **code serveur**. Le navigateur n'accède **jamais** directement à la base ; ce sont les **routes serveur** (`+server.ts`) qui interrogent PostgreSQL via **Drizzle**.



2. Mise en route

2.1 Lancer le site

① Installer les dépendances

```
cd SavoraWeb  
npm install
```

② Configurer la connexion à la base (fichier `.env`)

```
DATABASE_URL="postgresql://utilisateur:motdepasse@localhost:5432/savora"
```

③ Démarrer le site

```
npm run dev → http://localhost:5173
```

2.2 Visualiser la base — Drizzle Studio

Drizzle fournit une interface web pour **voir et modifier la base**. On la lance dans un terminal dédié :

```
npm run db:studio → https://local.drizzle.studio
```

On y voit les trois tables (`reservations` , `clients` , `daily_quotas`) et leur contenu.

À montrer au jury : faire une réservation sur le site, puis rafraîchir Drizzle Studio → la ligne apparaît. C'est la preuve que Drizzle enregistre bien en base.

3. Structure du projet

Fichier / dossier	Rôle
src/routes/+page.svelte	Page d'accueil (assemble la vitrine et le module de réservation)
src/lib/components/ (Navbar, Presentation, Philosophy, Manifeste, DiningHours, Footer)	Les sections de la vitrine
src/lib/components/Reservation.svelte	Le module de réservation (formulaire + disponibilités)
src/routes/api/reservations/+server.ts	Route serveur : enregistre une réservation (via Drizzle)
src/routes/api/availability/[date]/+server.ts	Route serveur : calcule les places restantes (via Drizzle)
src/lib/server/db/schema.ts	Description des tables (schéma Drizzle)
src/lib/server/db/queries.ts	Fonctions d'accès à la base (Drizzle)
src/lib/server/db/index.ts	Connexion à PostgreSQL

4. La vitrine

La vitrine présente le restaurant. Elle est découpée en composants réutilisables, assemblés dans la page d'accueil :

Composant	Contenu
Navbar	Barre de navigation en haut
Presentation	Présentation et image du restaurant
Philosophy	La philosophie / le concept
Manifeste	Le texte de présentation (manifeste)
DiningHours	Les horaires d'ouverture
Reservation	Le module de réservation (voir § 5)
Footer	Bas de page (localisation, contact)

5. Le module de réservation

C'est le cœur du site. Le client choisit, dans cet ordre :

- le **nombre de couverts** (1 à 6) ;
- la **date** (calendrier) ;
- l'**horaire** (midi ou soir) ;
- puis remplit ses **coordonnées** et valide.

5.1 Disponibilités en temps réel

Quand le client change de date, le site demande les places restantes à la route `/api/availability/{date}`. Les créneaux **complets sont grisés** et ne peuvent pas être sélectionnés. Cet affichage se met à jour automatiquement grâce à la réactivité de Svelte 5.

5.2 Enregistrement de la réservation

À la validation, le site envoie la demande à `/api/reservations`. Cette route serveur vérifie les règles (couverts 1–6, date non passée, conditions acceptées, quota non dépassé), puis enregistre la réservation **via Drizzle**.

Important : aucun e-mail n'est envoyé. La réservation est enregistrée avec le statut « **en attente** » et un récapitulatif s'affiche à l'écran. C'est le personnel qui la validera ensuite (dans l'application de gestion).

6. Accès aux données avec Drizzle

Drizzle permet d'écrire des requêtes SQL **en TypeScript**, de façon **typée** (le code vérifie les noms de colonnes) et **sécurisée** (protection automatique contre les injections SQL).

6.1 Calcul des places restantes (lecture)

```
// queries.ts – additionne les couverts déjà réservés pour un service
export async function reservedCovers(date, period) {
  const [row] = await db
    .select({ total: sql`coalesce(sum(${reservations.partySize}), 0)` })
    .from(reservations)
    .where(and(
      eq(reservations.date, date),
      eq(reservations.mealPeriod, period),
      inArray(reservations.status, ['pending', 'confirmed'])
    ));
  return Number(row?.total ?? 0);
}
```

6.2 Enregistrement d'une réservation (écriture)

```
// +server.ts – insère la réservation, puis crée le client si besoin
const [created] = await db.insert(reservations)
  .values({ partySize, date, timeSlot, mealPeriod, ... , status: 'pending' })
  .returning();

if (!(await clientExists(email)))
  await db.insert(clients).values({ civility, firstName, lastName, phone, email });
```

Ainsi, une réservation crée **automatiquement** la fiche client si elle n'existe pas encore (le lien se fait par l'adresse e-mail).

7. La base de données

Base : savora | **Moteur :** PostgreSQL | **3 tables** (affichables dans Drizzle Studio).

7.1 Table « reservations »

Colonne	Type	Description
id	entier	Identifiant unique
party_size	entier (1–6)	Nombre de couverts
date / time_slot	date / texte	Jour et heure (ex : 19:30)
meal_period	énum	midi (lunch) ou soir (dinner)
civility / first_name / last_name	texte	Identité du client
phone / email	texte	Coordonnées (l'e-mail relie au client)
status	énum	en attente / confirmée / refusée / annulée

7.2 Table « clients »

Colonne	Type	Description
id	entier	Identifiant unique
civility / first_name / last_name	texte	Identité
phone	texte	Téléphone
email	texte (unique)	Identifie le client de façon unique
notes	texte	Notes internes

7.3 Table « daily_quotas »

Colonne	Type	Description
id	entier	Identifiant unique
date / meal_period	date / énum	Jour et service (midi/soir)
max_covers	entier	Nombre de couverts maximum (15 par défaut)

8. Déploiement

Le site est déployé sur un **serveur Debian** (machine virtuelle), avec PostgreSQL installé **nativement** sur le serveur.

① Base de données — créer la base et un utilisateur dédié (non-root)

```
sudo -u postgres psql
CREATE USER savora_user WITH PASSWORD 'motdepasse';
CREATE DATABASE savora OWNER savora_user;
```

② Construire et lancer le site

```
npm install
npm run build
node build
```

→ site sur le port 3000

③ Apache (reverse proxy) — le serveur web public redirige vers le site

```
ProxyPass / http://localhost:3000/
ProxyPassReverse / http://localhost:3000/
```

Le site est alors accessible depuis l'**adresse IP du serveur**.

8.1 Sécurité (l'essentiel)

Risque	Protection
Injection SQL	Drizzle génère des requêtes paramétrées (jamais de SQL « à la main »)
Accès à la base	Seul le serveur accède à la base, jamais le navigateur
Mots de passe	Rangés dans le fichier <code>.env</code> (hors du dépôt Git)
Privilèges base	Utilisateur PostgreSQL dédié non-root
Données invalides	Contrôles avant enregistrement (couverts, date, conditions)

Phrase simple à retenir : « Le site SvelteKit sert les pages et, côté serveur, écrit dans PostgreSQL avec Drizzle. Aucune donnée ne transite par le navigateur sans contrôle. »